答辩材料：AI 交叉质询评审团

用途：上台讲、现场演、被问住时的兜底。

本文件是**源文件**，docs/答辩材料-AI交叉质询评审团.txt 与 ...pdf 由 scripts/make_defense_docs.py 从本文件导出，改内容只改这里。

全程口径以本文件为准；数字全部来自实测，凡是估算的地方都标了（估算）。

一、一页速览

项	内容
项目名	AI 交叉质询评审团 —— 让 AI 评审之间也互相质询
一句话	学生提交方案，四位不同视角的 AI 评审审阅、互相接话当场反驳，最后产出一份"你没答好的问题清单"
赛题定位	自拟选题，已核对 A01~A31 目录，无重题；对标 A05（模拟面试）与 A12（智课），但从"单考官"推进到"多评审对抗"
技术栈	FastAPI + LangGraph（多智能体图）+ PostgreSQL（LangGraph 检查点）+ Redis（会话/窗口记忆/画像）+ MySQL（质询记录）+ Chroma（长期记忆）+ Ollama 本地模型 + 前端单页（Vue3 + marked，无构建）
模型策略	正式演示走商业模型 qwen3.7-flash；本地兜底固定 qwen2.5:7b（换 qwen3.5:9b 会挂，实测见第七节）
完成度	M1~M5 全部已验收：方案上传/要素抽取 → 四位评审 → 交叉质询（SSE 流式）→ 学生作答 + 追问 → 会议纪要 + 未答好清单
已有工程增量	登录页（邮箱 + 验证码）、单页双智能体界面、按 session 删除会话（归属校验服务端判定）、三个可重跑的验收脚本（启动 19 项 + 页面 18 项 + 身份边界 15 项）
最终产出	会议纪要时间线 + 未答好的问题清单（按严重度降序）

二、电梯陈述

30 秒版（自我介绍后用）

我们做的是一个 **AI 交叉质询评审团**。

学生提交一份方案，系统里坐四个不同视角的 AI
评审——技术、成本与进度、合规与伦理、用户与价值。

关键不在于"四个 AI 各问各的"，而在于**评审之间会互相接话**：成本评审会打断技术评审——"你建议上大模型，可他方案里这个量级，预算撑不住"。

最后给学生的不是一句评语，而是一份**"你这几个问题没答好"**的清单。

2 分钟版（评委让你展开，或演示前铺垫）

先说我们解决什么问题。学生做项目，最缺的不是"写不出来"，而是**没人往死里问**。老师没时间逐份抠，同学之间问不到点上。市面上已有的 AI 面试产品，本质是**一个考官问你一串题**，你答得虚，它也只是换个题目接着问。

真实答辩现场最有杀伤力的从来不是"某个评委问了一个难题"，而是**评委 B 接过评委 A 的话**："你推荐的这套技术方案，他刚才说的预算根本撑不住。" —— 这是**评审之间的对抗**，现在的 AI 产品里没有。

所以我们把这件事做出来：给四个 AI

各自一套关注点，让每次发言广播给其他评审，谁认为"这话触碰到我的关注点"就**主动接话、当场反驳**。这是 agent 与 agent 之间的对抗，不是四个 AI 排队提问。

除了接话，还有两层：**判定**和**追问**。每一句问题都单独判"答了 / 答偏了 / 没答"，学生答得虚，原来问他的那位评审会**追一层**——追什么，取决于他刚才哪里没说到。

最后输出的不是分数，是一份**"未答好的问题清单"**，按严重度排序，每条都能点回当时的问答原文。这份清单可以直接当学生的修改工单用。

技术上我们还做了一件顺手但重要的事：**跨请求续接**。学生回答是另一次 HTTP 请求，会议状态存在 PostgreSQL 检查点里，进程重启都不丢——所以演示时中途刷新页面也接得上。

三、主线讲稿（5 分钟，逐字稿）

每段前面是掐表。**加粗**是重音位置，【】是动作提示。

语速按 220 字/分钟算，全文约 1100 字，正常讲 5 分钟。

0:00 - 0:40 开场：把痛点说成画面

【站定，不要急着点屏幕】

各位评委好。我先讲一个大家都见过的场景。

学生做完项目，把方案交上去，最怕的不是被骂，是**没人问**。老师带几十个学生，没法逐份抠；同学之间问来问去，也就是"你这个做得挺好的"。

而真正的答辩现场，最有杀伤力的从来不是某个评委问了一个特别难的问题。而是**评委 B 接过评委 A 的话**——"你推荐的这个技术方案，他刚才说的预算根本撑不住。"

评审之间的对抗，才是一份方案真正的压力测试。可今天所有的 AI 面试、AI 评阅产品，都只做到了"一个考官问你一串题"。

我们做的，就是把这件缺失的事补上。

0:40 - 1:40 核心：一句话讲清创新点

我们让 AI 评审之间也互相质询。

系统里坐四位评审，四个完全不同的视角：**技术评审**盯可行性，**成本与进度评审**盯预算和里程碑，**合规与伦理评审**盯数据来源和隐私，**用户与价值评审**盯是不是伪需求。

每位评审发言之后，这句话会**广播给其他三位**。其他评审判断："这句话有没有碰到我的关注点？"碰到了就**主动接话**——不是复述，是从**自己的立场**往下追。

所以我们的系统里会出现这样的对话：技术评审说"建议用大模型做语义匹配"；成本评审当场打断："你这个量级，光 API 成本每月就超预算三倍，这个方案不成立。"

【这里停一下，让评委反应过来】

这是 **agent 与 agent 之间的对抗**，也是我们和已有赛题 A05（单人考官）、A12（授课/RAG）最根本的区别。

1:40 - 3:30 演示：把最值钱的 30 秒现场还原

【切到屏幕，按第四节的操作顺序走】

我现在提交一份开题方案。【粘贴】

可以看到四位评审在几十秒内依次抛出四个**完全不同视角**的问题——技术问可行性、成本问花多少钱、合规问数据怎么来的、用户问到底谁会用。

【等接话出现，语速放慢】

接下来是这套系统最关键的 30 秒：请注意这条——成本与进度评审**接了技术评审的话**。它在说：你刚才建议的技术路径，在这个方案的预算下撑不住。这句话不是我们写死的模板，是模型自己判断出来"这碰到我的关注点了"。

【开始答题，故意答虚】

现在轮到我回答。我故意答一句空话——"这个我们后面会完善的，团队经验也比较足，您放心。"

请看判定：**没答到点上**。紧接着，刚才问我的那位评审**追了一层**，而且追的是我这句话里最虚的地方。

最后会议散场，产出两样东西：**会议纪要时间线**，每一句谁问的、我怎么答的、怎么判的、接的是谁的话，全在里面；以及这份 **"未答好的问题清单"**，按严重度从高到低排——最致命的两条排在最前面。

3:30 - 4:30 技术：只讲三个"这东西不好做"的点

讲三个我们真正花了功夫的地方。

第一，会议状态怎么续上。 学生的回答是**另一次 HTTP 请求**，中间可能隔几分钟。我们没有把会议状态放在内存里，而是存在 **PostgreSQL 检查点**里——进程重启、页面刷新，带着同一个 session_id 回来就接得上。这一点我们专门做过**杀进程验证**：起服务、开会、杀掉进程、重新起一个全新进程、再回答，会议正常往下走。

第二，怎么防止四个 AI 变成吵架或者跑偏。 三道锁：同一个议题里每位评审**最多接一次话**；整场会议接话**总数封顶**；同一个人不能连说。再加一道质量防线——接话发出来之前，和**整场会议里之前问过的每一句话**比语义相似度，超过 0.80 就先重试一次（还会告诉模型"你重复的是这一句"），仍不合格就**宁可放弃这次接话**，也不会把别人问过的话再说一遍。这个 0.80 不是拍脑袋定的：我们拿 8 场真模型评审会、26 条真实接话做过标定——原来的判据只看字面重复，26 条里只拦得住 1 条，换个说法就完全失效；换成向量余弦后拦得住 13 条。

第三，判定不是打总分。

一句话里塞三个问题，是判不出"答没答到点上"的，所以我们做了两件事：评审提问**只保留第一个问句**；判定**逐个问题做**。产出的是客观的"答了 / 答偏了 / 没答"，分数只作参考维度。

4:30 - 5:00 收尾：产出、差异、下一步

回到产出。我们最终给学生的不是一句"你的方案还行"，而是一份**可以当修改工单用的清单**：你有几个问题没答好，每一条都附着你当时的回答和判定理由。

和已有赛题比：A05 是**一个考官**问你一串题，A12 是**授课**；我们是**多位评审、可以互相质询**，最终产出是**未答好的问题清单**。已核对 A01~A31，无重题。

目前 M1 到 M5 已全部验收通过；下一步是把评审的长期画像接进来——同一个学生第二次被评审时，评审能记得他上次栽在哪。

我的介绍到这里，谢谢各位评委。

四、现场演示脚本

演示的目标只有一个：**让"接话"这 30 秒在现场真的发生，并且被评委看见。**

其余环节都是铺垫。所以下面每一步都写了"如果没出现怎么办"。

4.0 上台前 90 秒准备（第八节有完整清单）

1. 服务已起（`python -m app.main`，http://localhost:8000 ）
2. 浏览器**已经登录好**，停在主界面（登录要真发邮件验证码，现场不要赌邮件）
3. 提前打开 `data/demo_plan.txt`（706 字开题方案），复制到剪贴板
4. 缩放调到 100%，浏览器全屏，关掉无关标签页

4.1 步骤表

#	我做什么	我说什么	屏幕上应该看到
1	左侧点「交叉质询评审团」	"先看评审团这条线"	出现提交方案的输入区
2	粘贴 data/demo_plan.txt，点提交	"这是一份看起来写得挺完整的开题方案"	方案要素抽取中 → 出现要素表

#	我做什么	我说什么	屏幕上应该看到
3	不说话，等评审发言	"现在是四位不同视角的评审依次发问"	四个气泡依次出现，头像/标签各不相同（技术、成本、合规、用户）
4	停在接话那条上，指着屏幕	"请注意这条——成本评审接了技术评审的话"	气泡上带 → 接 XXX 的话 标记
5	在答题栏输入一句空话，回车	"我故意答得虚一点"	判定：没答到点上（严重度 3~4），并附理由
6	等追问出现	"刚才问我的那位评审追了一层"	带 └ 追问 标记的新气泡
7	再答一句（可继续敷衍）	"它追的是我这句话里最虚的地方"	判定 → 继续下一议题
8	一路答完（也可以选"跳过"）	"跳过它就不追了，追问一个已经承认不会的问题没有信息量"	逐条判定后散会
9	散会后滚到下方	"这才是最终交付物"	会议纪要时间线 + 未答好的问题清单（按严重度降序）
10	（可选）回「模拟面试」发一句	"同一套基础设施，也支持基础聊天和模拟面试"	模拟面试正常作答

4.2 三个"如果"

情况	现场怎么说 / 怎么做
接话一直没出现	别硬等。说："接话是评审自己判断触没触碰关注点，属于概率事件，本地小模型更保守；我换一场用商业模型的看看。" → 重开一场（重新提交方案）。上台前一定用商业模型预跑 2~3 次，确认接话稳定出现
评审说了重复的话，或角色串味	主动承认并解释机制："这正是我们加了语义去重防线（向量余弦 0.80，比对整场问过的每一句）的原因——重复就重试一次，还重复就宁可少一次接话。本地 7B 模型能力上限就在这，演示走的是商业模型。"
中途页面刷新 / 需要重启服务	直接说："学生回答是另一次请求，会议状态在 PostgreSQL 检查点里。" → 刷新后带着同一个会话继续答题，这就是最好的现场证明（我们做过杀进程验证：进程重启后仍能接上）

4.3 演示的三个看点（心里要有数）

1. **接话**（第 4 步）——立命之本，全场最值钱的 30 秒
2. **追问**（第 6 步）——证明"它真的听懂了我没答到点上"
3. **未答好清单**（第 9 步）——最终产出，把整场打分变成可执行的修改工单

五、预判提问与回答（20 条）

用法：加粗那行是**一句话答案**，后面是展开。答完看一眼评委，别自己往下加戏。

A. 选题与创新

1. 这和 A05 模拟面试有什么区别？不就是四个 AI 各问各的吗？

四个 AI 各问各的只是"四个考官"，我们要的是**评审之间的对抗。**区别有两处可验证：一是同一议题里接话**每人只允许一次**、全场封顶，说明我们把它当成需要控制强度的机制在做；二是接话带着**箭头信息**（谁接谁的话）落库并可回看——如果只是各问各的，这两个字段根本不会存在。

2. 交叉质询会不会是概率事件，现场不出现怎么办？

是概率事件，但我们把它做成了**可判定、可复现**的行为：接话要同时满足"和自身关注点相关""不和整场已问过的话语义重复（向量余弦 < 0.80）""本议题还没接过话""全场没超额度"。实测一场会 4 个主问题会带出最多 4 次接话，箭头方向全部正确。上台前我们用商业模型预跑，确保演示那次能出现。

3. 为什么必须用大模型？规则引擎/专家系统不能做吗？

"这句话有没有触碰我的关注点""学生这句回答算不算答到点上"，是**语义判断**，规则匹配做不了。我们**只在需要语义的地方用模型**：要素抽取、发言、判定。而调度、上限、清场、清单归拢这些**全是确定性代码**——这一点很重要，因为验收里有一整组"不调模型"的硬规则用例（8 个调度分支 + 5 条归拢规则），保证行为稳定。

B. 技术实现

4. 四个 AI 会不会最后说一样的话？

有实测数据。四位评审两两视角相似度**最高 0.07、平均 0.03**（越低越不重复），人设越界 0/4（没有评审问了自己不该问的），一问多问 0/4。做法是每位评审的关注点清单**写死且互相排他**，生成后还做一次去同质化检查。

5. 会不会变成四个 AI 无限互怼？

三道锁：**同一议题每位评审最多接话 1 次**、**全场接话总数封顶**、**同一议题已发言者不再接话**；再加一道语义去重防线（向量余弦 0.80，比对整场问过的每一句，重复就重试、还重复就放弃）。追问也有独立封顶：**每个议题最多追 1 层**（代码里是 `MAX_FOLLOWUP` 常量，改一个数就能变 2 层）。

6. 学生回答是另一次请求，会议状态怎么续上？进程重启会丢吗？

不丢。会议状态存 **PostgreSQL 检查点**（LangGraph 的 `AsyncPostgresSaver`），续接靠"同一个 thread_id 重新推流"。我们做过**跨进程验证**：起服务开会 → 杀进程 → 重新起一个全新进程 → 带同一个 session_id 回答 → 接口返回 200 而不是 404，判定事件正常回来，检查点数从 25 涨到 33。

7. 为什么评审会不用长期记忆？是偷懒吗？

不是，是**故意的边界**。一次评审会只活在一场会议里，需要的是"别忘了刚才说到哪"——那是检查点的事；把学生的长期画像喂进来反而会干扰判定（评审该按方案本身判，不该因为"这孩子上次也答得不好"而降低标准）。所以我们把它讲成**一套记忆基础设施、两种用法**：模拟面试用四层记忆（Redis 窗口、PostgreSQL 摘要、Chroma 长期、Redis 画像），评审会只用检查点 + 质询记录落库。

8. 判定"没答到点上"会不会很主观？

我们把主观性压到最小：**判定逐个问题做**（一句话里多个问题就判不出来），输出的是三档客观结论——答了 / 答偏了 / 没答，附带**严重度和一句理由**；提示词里明确要求"严禁因为学生态度好就给 resolved"。分数不作为主产出，只保留客观判定和按严重度排序的清单。

9. 本地模型和商业模型是什么关系？

两条通道：正式演示与生产走商业模型 `qwen3.7-flash`，本地 Ollama 的 `qwen2.5:7b` 作为降级兜底。**本地模型锁定 `qwen2.5:7b`**——我们实测过换 `qwen3.5:9b`：抽一次要素要 **313.8 秒**（7B 只要 10~12 秒），而且它不支持原生结构化输出，会返回带 ```json 围栏的文本，解析必然失败。间歇性失败在答辩现场是致命的，所以不换。

10. 方案很长，超上下文怎么办？

不把整篇方案喂给评审。提交时先抽**方案要素表**（8 个字段：目标、场景、规模、技术路径、量化指标、竞品对比、用户验证、缺失项），评审**只基于要素表提问**，提示词里硬约束"只能用要素表里已有的信息"。这样 token 可控，也顺带修掉了小模型脑补方案里没有的内容的问题。缺失项那一栏还是**代码判定**的，不交给模型。

11. 并发能撑多少人？做过压测吗？

没做过压测，这是我们的短板（诚实说）。可以讲的结构性事实是：会议状态在 PostgreSQL 而不是进程内存，所以**服务可以多进程横向扩**；模型调用是外部瓶颈，本地 Ollama 单机并发很有限，正式部署应该走商业模型的并发额度。另外我们做过会话级异步锁，避免同一个会话被两个请求同时推。

C. 数据与合规

12. 学生提交的方案数据存在哪？存多久？有授权吗？

方案正文与要素表存 MySQL 的 `review_session`，质询记录存 `review_question`，会议状态存 PostgreSQL 检查点，会话与窗口记忆在 Redis 并有 TTL。**目前没有做面向学生的数据保留期承诺和授权流程，这是明确的缺口**——合规这一块我们做了评审角色，但产品自身的数据治理还没做，下一步要补。

13. 你们系统里那位"合规与伦理评审"，它自己合规吗？

问得好——这正是我们第五个议题的来源。现状是：数据落地在自有 MySQL/PostgreSQL，不做外部分享；但**没有脱敏、没有保留期开关、没有授权留痕**。我们不假装这些做完了。

D. 成本与进度

14. 每个月大模型调用要花多少钱？

我们没有实测账单，给的是**估算口径**（这是估算）：一场评审会大约 5 次 HTTP 请求、**116 个流式事件**、8 次发言（含接话）+ 4~8 次判定 + 1 次要素抽取，合起来**约 20 次模型调用**。按商业模型的价格算单场成本，再乘"每班多少人 × 每学期几份方案"就能推出量级。真正省钱的地方在于：**本地模型可以整场兜底**，高校场景下完全可以只在关键判定上走商业模型。

15. 这个东西做了多久？谁做的？

【按你的实际情况填：几个人、各自负责哪一块、总时长。】可以说的是**工作量是可数的**：M1~M5 五个阶段全部验收通过，另有三个可重跑的验收脚本（启动侧 19 项、页面侧 18 项、身份边界 15 项），每次改完都跑一遍。

16. 现在完成到什么程度？还有什么没做？

M1~M5 全部完成并验收：方案上传/要素抽取、四位评审、交叉质询（SSE 流式）、学生作答 + 追问、会议纪要 + 未答好清单。另外还做了登录页（邮箱 + 验证码）、单页双智能体界面、按会话删除数据。**没做的**：四维雷达图与导出（主动砍掉）、评审的长期画像（下一步）、压测（短板）、数据合规治理（缺口）。

E. 价值与落地

17. 谁会真的用？学生凭什么用它？

三个真实场景：**课程设计/毕设开题**（学生自己先被问一遍）、**竞赛路演前的模拟答辩**、**教师批量筛查**（老师看"未答好清单"就知道哪几份需要细看，不用逐份读）。它比"找个同学帮我看"强的点是：不欠人情、随时可用、**问的是他想不到的视角**——成本、合规这两个视角恰恰是学生最容易漏的。

18. 和市面上的 AI 模拟面试产品比呢？

那些产品被评估的对象是**人**（求职者本人），AI 是考官，产出是评分。我们被评估的对象是**方案/作品**，AI 是**多位可以互相质询的评审**，产出是**问题清单**。一句话：他们评人，我们评方案；他们一个人问，我们四个评审会互相打断。

19. 这套东西能不能脱离"方案评审"这个场景？

框架本身是通用的：只要能把"被评估的对象"抽成要素表，再给出**互相有张力的关注点**，就能套用。比如代码评审（正确性 / 性能 / 可维护性 / 安全）、论文开题、项目验收。**张力**是关键——四个视角如果互不冲突，接话就不会发生。

20. 你觉得这个项目最大的局限是什么？（这一条准备着，被问到说明评委认了）

三处，我都写进 README 的"已知事项"了：① **接话质量随模型波动**，规则和额度确定性可复现，但内容质量在 run 之间有波动，根因是本地小模型能力上限；② **老评审会没有归属信息** —— 身份与归属校验已经做到五个评审接口加删除（服务端从登录会话反查账号，`review_session.student_id` 说了算），但改动之前提交的会这一列是空的，"没主"的记录按幂等放行，知道 id 的人仍能读 / 删；③ **有两处只走商业模型、没有本地兜底**（画像与评估节点），key 失效时会硬报错。这三条我都宁可自己先说。

六、技术底牌速查（被追问实现细节时翻）

评委可能追问	直接可讲的事实
多智能体怎么实现的	LangGraph 图 + 显式状态机，会议阶段有 8 个取值：extract / main / cross / await_answer / judge / followup / advance / done
"停住等学生回答"怎么做的	不是挂起。await_answer 阶段直接 goto=END 结束本次请求，续接靠检查点 + 同一个 thread_id 重新推流
接口怎么分的	POST /review/meeting/start（开场）与 POST /review/meeting/answer（学生回答）两个入口，中途状态在检查点里
流式怎么给的	SSE，事件类型 9 种（meeting_start / elements / speaker_start / token / speaker_end / verdict / await_answer / meeting_end / done），实测一场 116 个事件，speaker_start 与 speaker_end 配对 8/8
谁判定？谁追问?	那条问题的提问者判定；追问也由没被说服的提问者发起（这个设计让"接话问了没人答"不再发生）
一个问题好几个人问，学生答哪个	按提问顺序依次答，屏幕上永远只有一条"待你回答"，避免"三个人同时问、我不知道答谁"
未答好清单怎么保证不错	不单独存表，从质询记录推出来——推出来的永远和判定一致，"追问答好了就不算未答好"这条规则自动生效
数据落在哪	review_session（8 字段）+ review_question（13 字段）在 MySQL；会议检查点在 PostgreSQL；会话/窗口记忆/画像在 Redis
会话能删干净吗	DELETE /session/{id}：Redis 三个键 + PostgreSQL 摘要 + LangGraph 检查点 + MySQL 两张表，一次清完。验收是先造齐痕迹再删、逐样数回 0，不是只看接口返回 200
怎么保证改动不破坏功能	三个可重跑的验收脚本：scripts/check_startup.py 19 项（真起服务）、scripts/check_page.py 18 项（真读页面）、scripts/check_auth_boundary.py 15 项（身份与归属，离线）；身份那套还有"不带凭据 401 / 冒名不生效 / 别人的会读不了删不了"这种反向用例在盯
演示端口冲突怎么办	验收脚本自己起服务跑在 8801（CHECK_PORT 可改），不和你正在演示的 8000 抢

七、实测数据速查表

下面每一个数字都来自实测记录，全部口径与 docs/选题方案-AI交叉质询评审团.md 一致。

M1 方案提交与要素抽取

项	结果
两张表	review_session 8 字段、review_question 13 字段（建表脚本可重复执行）
文档解析	txt 705 字 / docx 699 字 / pdf 503 字，关键内容均命中
接口	upload、submit、session/{id} 全部 200，读回与写入一致
错误输入	空方案 / 旧版 .doc / 空文件 / 不存在的会话 → 400 / 400 / 400 / 404，没有 500

M2 四位评审与视角去同质化

指标	结果
出题数量	4 / 4
两两视角相似度	最高 0.07，平均 0.03
人设越界	0 / 4
一问多问	0 / 4

M3 交叉质询（SSE）

指标	结果
场景 A（关掉接话）	四位按 tech → cost → compliance → user 各发言一次，0 次接话，会议正常结束
场景 B（打开接话）	4 个主问题 + 最多 4 次接话，箭头方向全部正确
场景 C 事件统计	5 次请求 / 116 个事件：meeting_start 1、elements 1、speaker_start 8、token 84、speaker_end 8、verdict 4、await_answer 4、meeting_end 1、done 5
配对与箭头	speaker_start/end 配对 8/8；接话事件带箭头 4/4

M4 学生作答 + 追问

场景	结果
A：一路跳过（5 次请求 / 108 事件）	判定全为 unresolved，0 次追问，未答好清单 4/4
B：四次都答同一句空话	4 个议题全部开火追问，且追问接着空话往下扎；封顶 1 层，无套娃
C：不调模型的硬规则	8 个调度分支 + 5 条清单归拢规则全部通过

M5 会议纪要与未答好清单

检查项	结果
散会落库	meeting_end 带纪要正文与清单（7 条记录 / 4 个未答好），状态改 finished
库与事件一致性	库 7 条 / 事件 7 条，字段逐条一致（含 target_speaker、followup_depth、verdict_comment）
清单	按严重度降序，每条都能在纪要里找到出处
DAO 单测	交叉质询 target_speaker='cost'、追问 followup_depth=1、严重度 [4, 4]；重开是替换不是叠加

本地模型选型（为什么不换 qwen3.5:9b）

现象	数据
默认带思考，正文被挤空	抽一次要素 313.8 秒（7B 10~12 秒），一句话提问 5.9 秒，正文返回空串
关思考的开关不同	enable_thinking 无效（那是 DashScope 参数）；reasoning_effort: "none" 有效
不支持原生结构化输出	response_format 返回带 json 围栏的文本，解析必然失败

结论：本地模型固定 `qwen2.5:7b`，正式演示走商业模型 `qwen3.7-flash`。

与已有赛题的差异

	A05 模拟面试	A12 智课	本方案
被评估的对象	求职者本人	学生听课效果	学生的方案 / 作品
AI 的角色	考官（单人）	授课者	多位评审（可互相质询）
核心机制	题库驱动	RAG + 知识点树	多智能体交叉质询
最终产出	面试评分	讲稿 + 音频	未答好的问题清单

八、上台前检查清单与故障兜底

8.1 环境（提前 15 分钟过一遍）

#	检查	怎么做
1	Redis 在跑	6379 能连上（会话、窗口记忆、画像都在这）
2	PostgreSQL 在跑	会议检查点在这；POSTGRESQL_URL 在 .env
3	MySQL 在跑	评审会两张表在这
4	Ollama 在跑	11434 在监听（本地兜底模型）
5	商业模型 key 有效	DASHSCOPE_API_KEY 在系统环境变量里
6	服务能起来	python -m app.main → 看到两行"启动成功"，浏览器打开 http://localhost:8000
7	页面是最新的	浏览器 Ctrl+F5 强刷一次
8	跑一遍验收	python scripts/check_startup.py（19 项，跑在 8801，不占演示端口）
9	演示素材	data/demo_plan.txt 已复制到剪贴板
10	提前登录	登录要真发邮箱验证码，现场不要赌邮件；提前登好，别点"退出"

8.2 三个真实风险与兜底

风险	现场兜底
邮件验证码收不到（登录真发信，没有兜底码）	提前登录、不要退出（登录状态存在浏览器里）；终极兜底：备一段录制好的演示视频/截图，说"环境网络原因，我用录屏走一遍流程"
接话没出现	重开一场（重新提交方案）；用商业模型；讲解"这是概率性行为，我们加了额度与相似度控制"
模型超时/网络断	说明降级链路：商业模型失败 → 自动切本地 Ollama → 会议照开。本地慢，但对评委讲"降级可用"本身就是加分项

8.3 三处诚实边界（被问到就直说，别绕）

1. **接话内容质量随模型波动**：机制、调度、额度、箭头全部可复现；内容质量在 run 之间有波动（有时不理对方、偶尔角色串味），根因是本地 7B 模型能力上限。

2. **老评审会没有归属信息**：身份与归属校验现在已经做到五个评审接口加删除（`app/web/identity.py` 收口，`review_session.student_id` 说了算）；但**改动之前提交的会这一列是空的**（29 行、非空 0 行），这种"没主"的记录按幂等放行 —— 知道那个 32 位 hex 的人仍能读 / 删它，猜是猜不到。要彻底收掉得给存量数据补一次归属。

3. **两处只走商业模型、没有本地兜底**：画像节点与评估节点。key

失效时会硬报错（画像那处是静默失败）。

九、一句话备查（自我介绍 / 结尾都能用）

我们做的是**让 AI 评审互相质询**：四位不同视角的评审审一份方案，谁觉得碰到自己的关注点就当场接话反驳；学生答得虚会被原提问者追问一层；最后交出一份**未答好的问题清单**。

它不是"四个 AI 各问各的"，而是 **agent 与 agent 之间的对抗**——这是 A05（单人考官）和 A12（授课/RAG）都没有的东西。